

moe-engine: A Fault-Tolerant Runtime for Hyperscale Mixture-of-Experts Training

Min Htet Myet

Independent Researcher

github.com/Mattnal/Composed-Mixture-of-Experts-Engine

Abstract—We present moe-engine, a research-grade infrastructure runtime for training large Mixture-of-Experts (MoE) models under continuous node failure at hyperscale. The system combines a fused Triton routing kernel, composable 4D parallelism (data, expert, tensor, pipeline), invariant-checked correctness, and elastic fault tolerance with automatic expert resharding. This paper reports two developments beyond our prior work: first, the system’s first sustained validation on real GPU hardware (NVIDIA T4), which surfaced and corrected a kernel compilation defect invisible to CPU-only continuous integration and yielded measured throughput results including an $80.1\times$ speedup over a CPU reference path and a 100% pass rate on a storage-fault chaos scenario; second, a structural refactoring replacing a 1,165-line monolithic module with eight single-responsibility components, a migration to statically validated configuration, and a Switch-Transformer-style expert capacity enforcement mechanism. We additionally report, as a methodological contribution, a root-caused case study of a configuration validator that silently degraded to a no-op across an entire release cycle due to an undeclared dependency — a failure mode we argue generalizes to any system that pairs safety-critical validation with an optional-dependency fallback. The test suite grew from 148 to 348 passing cases over this arc, incorporating property-based testing and a mocked collective-communication backend. We report multi-node GPU benchmarking and full chaos-recovery reliability as open work, consistent with this project’s stated policy of reporting measured results only at the scale actually validated.

Index Terms—Mixture-of-Experts, distributed training, fault tolerance, GPU kernels, Triton, elastic training, expert parallelism, configuration validation

I. INTRODUCTION

Training large Mixture-of-Experts (MoE) models at hyperscale — tens of thousands of GPUs — exposes systems challenges beyond raw throughput: continuous node failure, silent collective and sharding bugs, misleading utilization metrics under sparse activation, and the need for seamless recovery with expert re-ownership [1], [2]. moe-engine is a runtime built specifically around the premise that at hyperscale, node death is not an exception to be handled but a continuous background condition the system must remain correct under.

This paper is the third report on the system. The first version described the initial Triton router kernel, 4D mesh foundations, and elastic recovery mechanisms. The second reported production-style multi-process pipeline parallelism with a full 1F1B schedule, but explicitly declined to present GPU throughput data, stating that sustained cluster access was not yet available and that the authors preferred to report no numbers rather than illustrative ones. This paper reports the

closing of that specific gap at single-GPU scale, together with a substantial internal refactoring undertaken in parallel.

Our contributions are:

- The first real-GPU validation of the system’s Triton router kernel, which surfaced a compilation defect (Section IV) invisible to any CPU-only test, together with measured throughput data (Section X).
- A structural decomposition of a 1,165-line distributed-systems module into eight single-responsibility components with full backward compatibility (Section III).
- An expert-capacity enforcement mechanism following Switch Transformer/GShard semantics [3], [4], implemented via a vectorized group-cumulative-count primitive (Section VI).
- A root-caused case study of a silent configuration-validation failure with a generalizable lesson for systems pairing optional dependencies with safety-critical logic (Section VII).
- An expanded test suite (148→348 tests) incorporating property-based testing and mocked collective communication (Section IX).

II. RELATED WORK

Megatron-LM [1] and DeepSpeed [2] established tensor- and pipeline-parallel training at scale for dense models. Switch Transformer [3] and GShard [4] introduced sparse expert routing with capacity-based token dropping, which we adopt directly for our capacity enforcement mechanism (Section VI). Tutel [5] addressed adaptive expert parallelism at scale. Prior systems in this space have generally reported results at the multi-node scale their infrastructure was designed for; our project differs primarily in scope and maturity level — it targets research-grade reproducibility on resources ranging from a single CPU core to, eventually, a full cluster, and explicitly reports which scale each result was obtained at rather than presenting cluster-scale projections without cluster-scale data.

III. SYSTEM ARCHITECTURE

The system’s high-level control flow — training loop, distributed MoE layer, elastic training harness, telemetry — is unchanged from prior versions. The principal architectural change in this work is a decomposition of the distributed-systems package, previously a single 1,165-line file containing every parallelism primitive, into eight modules (Table I),

TABLE I
DISTRIBUTED MODULE DECOMPOSITION

Module	Responsibility
mesh.py	Topology, device mesh, comm. streams
tensor_parallel.py	Column/Row-parallel linear layers
sequence_parallel.py	SP scatter/gather, fused all-gather
expert_parallel.py	All-to-all dispatch/combine
pipeline_parallel.py	1F1B pipeline schedule
data_parallel.py	FSDP2 application
router.py	High-level router interface
moe_layer.py	Layer orchestration

each with one responsibility. A 68-line backward-compatibility shim re-exports every public symbol from the original module path, so existing call sites require no modification.

Model definitions were additionally extracted into a separate package with a decorator-based registry (`register_model`, `build_model_from_config`), following the pattern used by Megatron-LM and torchTitan to decouple training infrastructure from model research code: the training entrypoint dispatches on a configuration string rather than importing a specific model class.

IV. TRITON ROUTER KERNEL

The router kernel fuses gating matmul, softmax, top- K selection, and combine-weight renormalization into a single Triton kernel operating in one HBM pass, unchanged architecturally since our first report. This section describes a defect surfaced only when the kernel was, for the first time, compiled and executed on real GPU hardware as part of this work.

A. The *constexpr* Defect

Both the forward and backward kernels unroll their top- K selection loop using `tl.static_range`, which requires the loop bound to be a compile-time *constexpr* value. The kernel signatures declared K as an ordinary runtime integer. On the CPU-only reference implementation used throughout prior continuous integration — plain PyTorch, never invoking the Triton compiler — this had no observable effect. The first real-GPU compilation attempt failed with a `TritonCompilationError`.

We highlight this defect because of its generalizable failure mode: *a CPU reference fallback for hardware-specific code is exactly what allows a hardware-only defect to survive an arbitrary number of CI runs*, because CI never exercises the code path where the defect resides. The fix — declaring K as *constexpr* in both kernel signatures — is minor; the broader lesson motivated two regression tests: a CPU-executable static assertion that both kernel signatures declare K as *constexpr*, and a GPU-gated test that compiles and runs the kernel when CUDA is available.

V. 4D PARALLELISM

Data, expert, tensor, and pipeline parallelism compose via a device mesh constructed once at startup. Tensor parallelism shards SwiGLU expert weights following the stan-

Algorithm 1 `cumcount`: position of appearance within group

Require: $\text{groups} \in \mathbb{Z}^N$ (expert assignment per token)

Ensure: $\text{position} \in \mathbb{Z}^N$ (0-indexed rank within group)

- 1: $\text{order} \leftarrow \text{STABLEARGSORT}(\text{groups})$
- 2: $\text{sorted} \leftarrow \text{groups}[\text{order}]$
- 3: $\text{isStart}[0] \leftarrow \text{true}; \text{isStart}[i] \leftarrow \text{sorted}[i] \neq \text{sorted}[i-1]$
- 4: $\text{startIdx} \leftarrow \text{WHERE}(\text{isStart}, i, 0)$
- 5: $\text{startIdx} \leftarrow \text{CUMMAX}(\text{startIdx})$
- 6: $\text{posSorted} \leftarrow i - \text{startIdx}$
- 7: $\text{position}[\text{order}] \leftarrow \text{posSorted}$
- 8: **return** position

dard column/column/row convention (both gate and up projections column-parallel, down projection row-parallel), unchanged from prior work. Pipeline parallelism uses a three-phase 1F1B schedule (warmup, steady-state, drain) with real `torch.distributed.send/recv` on a dedicated process group and explicit micro-batch tagging, validated by 2-rank `mp.spawn` tests; this component is unchanged in the present work and has not yet been exercised under simulated node failure.

VI. EXPERT CAPACITY MANAGEMENT

The principal functional addition in this work is an optional expert capacity enforcement mechanism following Switch Transformer/GShard semantics: each expert accepts at most

$$c_e = \left\lceil \gamma \cdot \frac{NK}{E} \right\rceil \quad (1)$$

tokens, where γ is a configurable capacity factor, N is the token count, K is active experts per token, and E is total expert count. Tokens beyond this budget receive zero combine weight for the corresponding top- K slot rather than being processed.

Algorithm 1 gives the core primitive, `cumcount`, which computes each token’s zero-indexed position of appearance within its assigned expert’s queue using a stable sort followed by a cumulative maximum, avoiding both an explicit per-expert loop and a dependency on `scatter_reduce` variants whose availability differs across PyTorch releases. The capacity drop mask is then $\text{drop}[n, k] = \text{cumcount}(\text{idx}[:, k])[n] \geq c_e$, applied independently per top- K slot. The mechanism is disabled by default; enabling it changes no existing behavior for callers who do not opt in. Twenty-two dedicated tests cover the primitive, mask construction, and full-layer integration including gradient flow through non-dropped slots.

A companion configuration exercising $E=256$, $K=8$ — a fine-grained expert regime four times the expert count of our existing production configuration — validates the mechanism’s correctness at reduced dimensions; its throughput and memory behavior at full scale on real hardware has not been measured (Section XI).

VII. CONFIGURATION VALIDATION

A. Schema Migration

Prior configuration handling used an unvalidated flat dictionary: an invalid value such as `top_k > num_experts` would

TABLE II
SILENT-DEGRADATION DEFECTS AND THEIR FIXES

Root cause	Fix
Validation library never declared as a runtime dependency; optional-import fallback silently disabled all validation	Declared as a hard dependency; fallback path deleted entirely — module now fails loudly on import if absent
<code>yaml.safe_load("1e-5")</code> returns a string under YAML 1.1 grammar, not a float	Native <code>int/float</code> coercion attempted before YAML fallback

not surface until the router kernel failed with an opaque index error, potentially minutes into training. We replaced this with a statically validated hierarchical schema with cross-field validators enforcing, among other constraints, $\text{top_k} \leq \text{num_experts}$ and $\text{warmup_steps} < \text{max_steps}$, reporting the offending field path and constraint description at load time.

B. A Silent-Degradation Case Study

We report the following defect in detail for its methodological interest. The configuration module included a fallback: if the validation library were unavailable, the module would degrade to an unvalidated pass-through rather than fail to import, intended to preserve importability in minimal environments. The dependency, however, was never declared in the package’s runtime requirements. Continuous integration installed exactly what was declared, so the validation library was never present, and every configuration — including deliberately invalid ones constructed by the test suite specifically to verify that validation errors fire — silently passed through the unvalidated path. Eight assertions that a specific exception type would be raised instead observed no exception, for an entire release cycle, without any test reporting failure, because the tests never exercised the code path they were written to test.

The general lesson (Table II) we draw is that *a component whose stated purpose is catching invalid input early should not be able to silently stop performing that function*; an import error is a safer failure mode than a silently permissive one. We deleted the fallback path entirely rather than fixing the dependency declaration alone.

VIII. FAULT TOLERANCE AND CHECKPOINTING

Asynchronous two-tier checkpointing (pinned host memory, NVMe with `O_DIRECT` and atomic rename, S3) and round-robin expert resharding on node loss are unchanged in core algorithm from prior work. We added explicit schema versioning to checkpoint metadata, embedding the runtime’s package and PyTorch versions (read from their respective single sources of truth, not hardcoded) alongside a monotonically increasing schema version integer, with graceful field defaulting on version mismatch in either direction. The storage-stall chaos scenario remains fully verified and, as reported in Section X, now has real-GPU confirmation. The node-kill-and-recovery scenario is unchanged at approximately 85% pass rate due to a Gloo process-group re-initialization race, whose fix (NCCL

TABLE III
ROUTER FORWARD THROUGHPUT: T4 GPU VS. CPU REFERENCE

N	H	E	K	CPU (M/s)	GPU (M/s)	Speedup
512	256	16	2	0.747	2.141	$2.9\times$
1024	512	32	2	0.421	3.644	$8.7\times$
2048	1024	64	2	0.236	4.832	$20.4\times$
4096	2048	64	4	0.056	4.454	$80.1\times$

migration in the chaos harness) requires GPU hardware not available for this cycle.

IX. TESTING INFRASTRUCTURE

The CPU-only test tier grew from 148 to 348 passing tests over this arc. Three additions specifically target correctness properties that do not require GPU or multi-process resources: (i) nine Hypothesis-based property tests verifying token conservation, weight normalization, complete and non-overlapping expert ownership under arbitrary $(E, \text{ep_size})$, and configuration invariants, each checked against fifty randomly generated inputs rather than fixed examples; (ii) a thread-based mock of `all_to_all_single` enabling expert-parallel dispatch logic to be exercised at $\text{ep_size} > 1$ within a single process, complementary to (not a replacement for) existing 2-rank `mp.spawn` ground-truth tests; and (iii) dedicated test suites for the high-level router interface and model registry (33 and 20 tests respectively), following the principle that each public interface boundary warrants tests that would fail if that boundary’s contract were violated, independent of underlying-implementation coverage.

X. EVALUATION

A. Real-GPU Validation

We report results from a single NVIDIA T4 (16 GB HBM2, 65 TFLOPS FP32), the first sustained GPU validation for this system. Token conservation held with zero violations across 100 seeds on CUDA, matching the CPU reference. The storage-stall chaos scenario achieved a 10-of-10 (100%) pass rate under real-GPU execution.

Table III reports forward-only router throughput at four configurations. Speedup increases super-linearly with problem size, from $2.9\times$ at $N=512$ to $80.1\times$ at $N=4096, H=2048$ — the configuration closest to production MoE hidden dimensions. We attribute this to the single-HBM-pass kernel’s bandwidth utilization improving with working-set size, unmatched by the single-threaded, unvectorized CPU reference path.

We emphasize scope: these results confirm kernel correctness and CPU/GPU scaling on real hardware for the first time, but do not constitute production-scale MFU. The T4 is inference-oriented, with roughly $15\times$ lower BF16 peak throughput than an H100 SXM5, and measured MFU at smoke-test batch sizes is on the order of 10^{-6} — expected and uninformative at this scale, reported for completeness rather than as a substantive result.

B. MoE Layer Overhead

Full distributed-layer forward latency, measured against a dense single-expert baseline at matched (B, S, H, F) , showed $21.8\times$ to $26.1\times$ overhead on the T4 and $9.1\times$ to $16.2\times$ on CPU at small batch sizes ($B \leq 4$). This range is dominated by kernel-launch latency at these sizes and by a parameter-count mismatch between the dense baseline and the MoE configuration compared against it; it should be read as a latency-floor measurement rather than a throughput-per-parameter comparison.

XI. LIMITATIONS AND FUTURE WORK

Consistent with this project’s reporting policy, we state open items explicitly rather than eliding them. *Multi-node GPU data* does not yet exist beyond the single-T4 results above; this remains the top priority for future work and requires sustained cluster access. *Chaos Scenario A* (node kill and recovery) remains at approximately 85% due to the unresolved Gloo race noted in Section VII; its fix requires GPU hardware to validate. The *expert capacity mechanism* of Section VI is validated for correctness only at reduced scale; its behavior at $E=256$ under real dispatch bandwidth constraints is unmeasured, as is a next-best-expert re-routing policy as an alternative to outright dropping. *Pipeline parallelism under simulated node failure* has not been exercised. *Non-divisible sequence lengths* under sequence parallelism remain unsupported.

XII. CONCLUSION

This paper reports closing a specific, previously stated gap: real-GPU validation of moe-engine’s router kernel and chaos-recovery mechanism, with reproducible single-device measurements rather than illustrative estimates. That validation surfaced a genuine defect invisible to any CPU-only test, illustrating a general risk in systems that maintain hardware-specific code behind a CPU fallback. In parallel work, a structural refactoring surfaced a second, independent defect — a configuration validator silently non-functional for a full release cycle due to an undeclared dependency — which we argue generalizes beyond this system to any component pairing safety-critical validation with an optional-dependency fallback. The system’s multi-node evaluation gap remains open for the reason it has been open throughout this project’s reporting history: access to sustained cluster resources, not algorithmic readiness.

ACKNOWLEDGMENT

The author thanks the PyTorch, Triton, and distributed systems communities, the maintainers of Pydantic and Hypothesis, and researchers who have openly shared large-scale training failure post-mortems.

REFERENCES

- [1] M. Shoyerbi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training multi-billion parameter language models using model parallelism,” *arXiv:1909.08053*, 2019.
- [2] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, “DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters,” in *Proc. 26th ACM SIGKDD Conf.*, 2020, pp. 3505–3506.
- [3] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *arXiv:2101.03961*, 2021.
- [4] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, “GShard: Scaling giant models with conditional computation and automatic sharding,” *arXiv:2006.16668*, 2020.
- [5] C. Hwang *et al.*, “Tutel: Adaptive mixture-of-experts at scale,” *arXiv:2206.03382*, 2023.